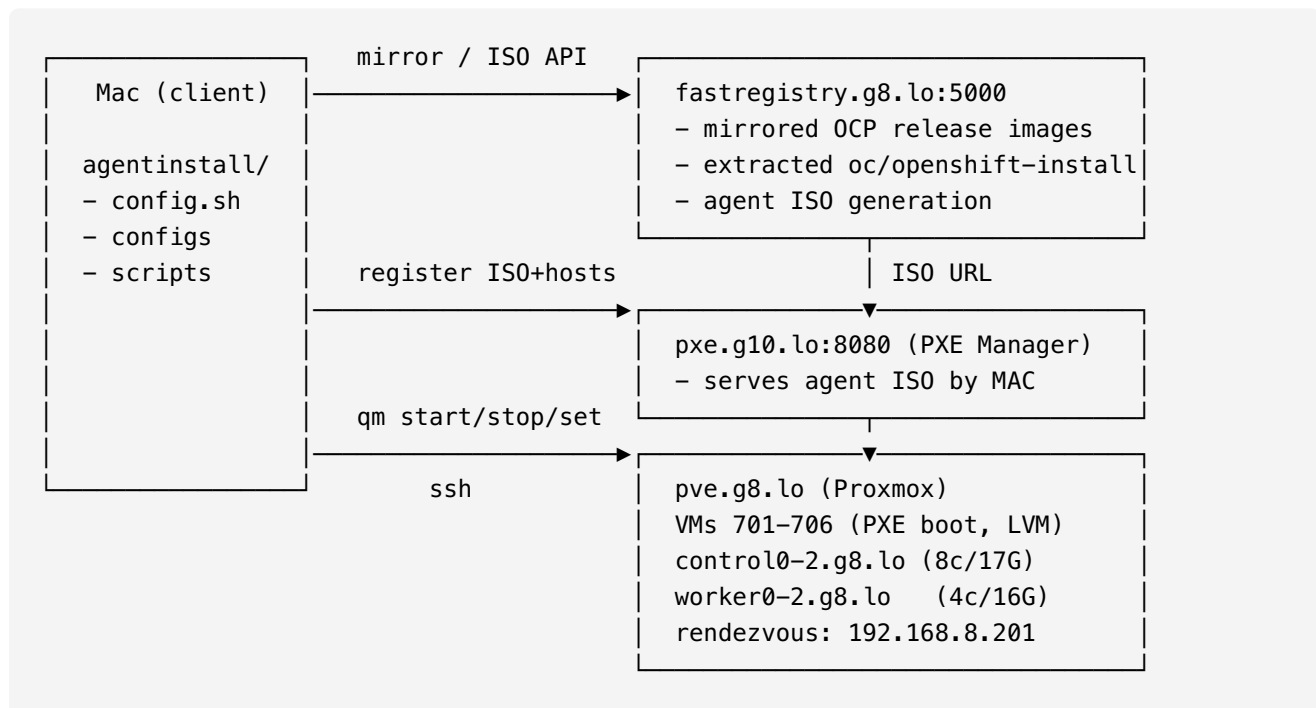


Agent-Based OpenShift Installation

Agent-based install of a 6-node OpenShift cluster (`g8` / `g8.lo`) as Proxmox VMs, driven entirely from this directory. Releases are mirrored into **FastRegistry** (`fastregistry.g8.lo:5000`), the agent ISO is generated **server-side** by FastRegistry, and nodes **PXE-boot** that ISO via the PXE Manager — no ISO is ever attached to a VM or downloaded to the Mac.

See [ECOSYSTEM.md](#) for how this project relates to agent-monitor, fastregistry, pxemanager, and the bare-metal SNO flow.

Architecture



Prerequisites

- FastRegistry running at `fastregistry.g8.lo:5000` (HTTP, no auth)
- PXE Manager running at `pxe.g10.lo:8080`, serving boot for the `g8` VMs
- SSH root access to `pve.g8.lo`
- DNS for the cluster (`api.g8.lo`, `*.apps.g8.lo`, node records)
- `oc`, `jq`, `python3` (with `pyyaml`) on the Mac
- `pullsecret.json` in this directory (gitignored)
- `install-config.yaml` created from `install-config.yaml.template` (gitignored)

Quick Start

```
./generate-secrets.sh    # first time only – writes .env
./create-vms.sh          # first time only – creates VMs 701–706
./mirror.sh 4.18         # mirror release into FastRegistry (resolves latest 4.18.x)
./install.sh 4.18        # full install, ~same version resolution
```

Version arguments accept `4.18.10` (exact), `4.18` or `4.18.z` (resolved to the latest 4.18.x by querying Quay.io release tags).

What `install.sh` Does

1. **Stop all VMs** (701–706) and verify they are stopped.
2. **Pre-flight** — verify the release image exists in FastRegistry (`oc image info`); tells you to run `./mirror.sh` if not.
3. **Pull openshift-install** via `pull-from-registry.sh` : already-installed version → `bin/` cache → FastRegistry file download → `oc adm release extract` fallback. Installs to `~/.local/bin`.
4. **Prepare workdir** — fresh `g8/` directory with `install-config.yaml` + `agent-config.yaml` ; run `openshift-install agent create config-image` locally (fast — produces kubeconfig and the state file for the wait-for commands, no ISO download).
5. **Generate the agent ISO on FastRegistry** — POST both YAML configs to `/admin/releases/<ver>-x86_64/iso` ; FastRegistry embeds the ignition into its cached base ISO and returns a URL (saved to `g8/.iso_url`).
6. **Wipe and verify disks** — zero each VM's LVM disk (`production-lvm/vm-<id>-disk-0`) and verify sector 0 is clean; set boot order to `net0;scsi0` and remove any attached IDE ISO.
7. **Register with PXE Manager** — add the ISO under a unique name (`ocp-<ver>-<timestamp>`) and register each host MAC/hostname from `agent-config.yaml` to boot it.
8. **Install kubeconfig** to `~/.kube/config` .
9. **Power on all nodes** — they PXE-boot the agent ISO; the rendezvous node (`192.168.8.201` = control0) coordinates the install. The Tk GUI monitor (`monitor.py`) launches in the background.
10. **Wait for bootstrap** — first watches bootkube on the rendezvous node for the `missing operand kubernetes version` crash-loop (a mismatched openshift-install/ISO — abort early instead of hanging), then runs `openshift-install agent wait-for bootstrap-complete` .
11. **Apply the MachineConfig desync fix** (`fix-mc-desync.sh`) as soon as the API answers — see [MC_BOOTSTRAP_DESYNC.md](#) .
12. **Wait for install-complete**, record the run in `install-history.json` , and print the install history (every run is timed, start → end).

Configuration (`config.sh`)

All settings live in `config.sh` , including helper functions (`resolve_latest_version` , install-history recording).

Variable	Value	Purpose
<code>LOCAL_REGISTRY</code>	<code>fastregistry.g8.lo:5000</code>	Registry host (HTTP, no auth)
<code>LOCAL_REPOSITORY</code>	<code>openshift/release</code>	Release image repo

Variable	Value	Purpose
FASTREGISTRY_URL	http://fastregistry.g8.lo:5000	Admin/file API
PXE_MANAGER_URL	http://pxe.g10.lo:8080	PXE Manager API
PVE_HOST	pve.g8.lo	Proxmox host (ssh as root)
CLUSTER_NAME / BASE_DOMAIN	g8 / g8.lo	Cluster identity; workdir name
RENDEZVOUS_IP	192.168.8.201	control0; assisted-installer API host
CONTROL_VM_IDS / WORKER_VM_IDS	701–703 / 704–706	Proxmox VM IDs
CONTROL_CORES/MEMORY	8 / 17000	Control-plane VM size
WORKER_CORES/MEMORY	4 / 16000	Worker VM size
LVM_VG / LVM_STORAGE	production-lvm	Thick-provisioned VM disks

Node identity (hostnames, MACs, roles, rendezvous) is in `agent-config.yaml` ; cluster settings (networks, image sources, trust bundle) come from your `install-config.yaml` (start from the template).

Scripts

Script	Purpose
mirror.sh <ver>	Clone a release into FastRegistry via its admin API and poll progress; artifacts (oc, openshift-install, ISOs) are extracted server-side
remirror.sh <ver>	Wipe the existing mirror and re-mirror fresh
install.sh <ver>	Full install (steps above)
pull-from-registry.sh <ver>	Fetch <code>openshift-install</code> for the local OS from FastRegistry, with caching
create-vms.sh / delete-vms.sh	Create/remove VMs 701–706 (fixed MACs matching <code>agent-config.yaml</code>)
poweron-all.sh / poweroff-all.sh	Bulk VM power control
verify.sh <ver>	Run <code>verify-local.sh</code> on the registry host against the mirror

Script	Purpose
<code>watch-install.sh</code>	<code>watch oc get clusteroperators</code>
<code>approvecsr.sh</code>	Loop auto-approving pending CSRs (run in a second terminal)
<code>fix-mc-desync.sh</code>	Repair MCP status/annotation desync after bootstrap pivot
<code>gatherdebug.sh [dir]</code>	Collect logs/state from all six nodes + cluster into a debug bundle
<code>wipe_registry.sh</code>	Nuke registry storage (legacy — targets the old Quay layout on <code>registry.g8.lo</code>)
<code>generate-secrets.sh</code>	Create <code>.env</code> with a generated registry password
<code>lib/vm.sh</code>	Shared Proxmox helpers (power, disk wipe, PXE boot order, disk type)

Monitoring an Install

- **monitor.py** — local Tkinter GUI polling the assisted-installer API on the rendezvous node; started automatically by `install.sh`.
- **agent-monitor** — the web console (`http://agentmonitor.g10.lo`); add the cluster with rendezvous IP `192.168.8.201` . See [agent-monitor](#).
- `./watch-install.sh` and `./approvecsr.sh` for CLI-side watching.

Troubleshooting

- **Release image not found** → `./mirror.sh <ver>` first.
- **kube-apiserver crash-loop, missing operand kubernetes version** — the ISO was built with a mismatched `openshift-install`; `install.sh` detects this in the first 3 minutes and aborts. Re-pull the installer and rerun.
- **control0 stuck fetching ignition after bootstrap** — the MachineConfig bootstrap desync; `install.sh` applies `fix-mc-desync.sh` automatically, or run it by hand. Details: [MC_BOOTSTRAP_DESYNC.md](#) and [MC_ANNOTATION_FIX.md](#) .
- **Anything else** → `./gatherdebug.sh` collects journals, pod logs, and install state from every node into a timestamped bundle.

Install history for this machine is kept in `install-history.json` (version, start/end, duration, completed) and printed after each install.